

Customer Churn Prediction System

Complete Project Documentation & Interview Preparation Guide

Author: Venkata Sesha Sainath Biruduraju

Role: Software & AI Engineer

Project Folder: C:\projects\churn_project

Date: June 2026

Table of Contents

- 1. Project Overview
- 2. Project Architecture & Data Flow
- 3. Implemented Features
- 4. Technologies Used
- 5. Folder Structure
- 6. Backend Explanation
- 7. Frontend Explanation (Not Applicable)
- 8. Relational Database Usage
- 9. Machine Learning Pipeline
- 10. Deployment Configuration
- 11. Key Technical Learnings
- 12. Challenges Faced & Solutions
- 13. Resume Support Materials
- 14. Comprehensive Interview Preparation
- 15. Candidate Elevator Pitches (Technical & HR)

1. Project Overview

Purpose: This project implements a fully integrated end-to-end machine learning pipeline and production-ready REST API to predict subscriber churn for subscription-based industries.

Problem Statement: Subscriber retention is critical for telecom operators, as customer acquisition costs far exceed the costs of retention. Predicting which users are likely to unsubscribe allows businesses to proactively engage them with tailored retention offers.

Objective: Build a highly portable, stateless pipeline that preprocesses raw customer churn CSV data, stores cleaned files in SQLite, trains multiple Scikit-learn classification models, auto-selects the best model based on F1-Score, and exposes a Flask prediction web server.

Target Users: Business analysts, customer relationship managers, and microservice consumers.

Real-world Use Case: A customer support agent opens a client dashboard, triggering a background call to this API. The API instantly returns the customer's churn risk and probability, allowing the agent to offer targeted discounts if the probability is high.

2. Project Architecture & Data Flow

The application divides work between an offline ETL/Training pipeline and an online Inference API:

Data flow: Raw CSV data is preprocessed, mapped, and encoded in preprocess.py, then written into the SQLite database churn.db. The label encoders are serialized to encoders.pkl. Next, train.py reads from churn.db, performs splits, fits models, selects the top F1-Score performer, and saves the model to model.pkl and feature columns to feature_columns.pkl.

Request-response flow: The web server (Flask) loads all artifacts on boot. A client submits a POST request to /predict containing raw customer traits. The API validates the keys, standardizes numeric values, applies loaded label encoders to categoricals, re-aligns the columns, and returns a JSON payload detailing the prediction ('Yes' or 'No') and the probability.

AI/ML Agent Architecture: Note that no AI Agent framework, LangChain, semantic search, vector databases, or Large Language Models (LLMs) are used in this project. The system relies entirely on classical supervised classification.

3. Implemented Features

- **Automated Preprocessing & Imputation:** Automatically handles dirty strings, parses columns, maps churn labels, and fills numeric missing values with calculated medians.
- **Consistent Categorical Encoding:** Fits Scikit-learn LabelEncoders and serializes them, eliminating the risk of train-serve category index mismatches.

- **Multi-Classifer Training & Auto-Selection:** Compares Logistic Regression, Random Forest, and Gradient Boosting, automatically selecting and saving the model with the highest F1-Score.
- **Production-grade API Validation:** Validates all required client fields, returning standard HTTP 400 error messages for missing columns or unseen categorical strings.

4. Technologies Used

The stack consists of the following components:

Technology	Role / Usage
Python 3.12.6	Core programming language runtime.
Pandas & NumPy	Used to load CSV, clean values, handle types, and query tables.
Scikit-learn	Used for encoders, model training, split, and metric calculation.
SQLite3	Local SQL storage file (churn.db) used to hold preprocessed customer data.
Flask	Micro-web framework to run the REST API and handle HTTP routes.
Joblib	Used to save/load models, encoders, and column lists to pkl files.
Gunicorn	WSGI server for concurrent production requests on Render.

Explicit Exclusions: PostgreSQL, MySQL, MongoDB, Redis, FastAPI, Django, Docker, Kubernetes, JWT, API keys, target encoding, or cloud S3 buckets are **NOT** used in this project.

5. Folder Structure

The project is structured within C:\projects\churn_project as follows:

```
C:\projects\churn_project\  
■■■ data\  
■   ■■■ WA_Fn-UseC_-Telco-Customer-Churn.csv # Raw dataset  
■■■ preprocess.py # Clean & export data to SQLite  
■■■ train.py # Model training & metrics evaluation  
■■■ app.py # Flask prediction API  
■■■ test_api.py # API client integration script  
■■■ requirements.txt # Python dependencies list  
■■■ Procfile # Gunicorn process config for Render  
■■■ runtime.txt # Explicit Python version  
■■■ .gitignore # Prevents local cache/venv check-ins  
■■■ churn.db # SQLite database containing customers table  
■■■ model.pkl # Best model checkpoint (Logistic Regression)
```

```
■■■ feature_columns.pkl           # Ordered feature list for inference
■■■ encoders.pkl                 # Pickled LabelEncoders dictionary
```

6. Backend Explanation

preprocess.py:

- Loads dataset using Pandas.
- Preprocesses TotalCharges to numeric, replacing whitespace with NaNs, and imputing them using the training median.
- Standardizes features by fitting LabelEncoders on the object-type variables, writing them to encoders.pkl.
- Writes clean rows to database churn.db in table 'customers'. Runs verification queries.

train.py:

- Pulls rows from churn.db.
- Generates an 80/20 train/test split.
- Fits Logistic Regression, Random Forest, and Gradient Boosting models.
- Prints metrics and outputs model.pkl and feature_columns.pkl.

app.py:

- Exposes GET /health (returning status ok) and POST /predict (predicting churn label and proba).
- Performs structural checks to catch missing parameters (returns 400 Bad Request if features are absent).

7. Frontend Explanation (Not Applicable)

This project is a **backend-only** service. It does not implement any user interface, HTML web page, JavaScript component, mobile application, or dashboard. All client interactions occur programmatically via HTTP POST requests to the REST API, as demonstrated by the test script test_api.py.

8. Relational Database Usage

The project uses an embedded SQLite database stored as churn.db in the root directory. All cleaned records are saved into the table named **customers**.

Table columns: All 19 preprocessed numeric and encoded categorical features are saved as columns (e.g., tenure, MonthlyCharges, TotalCharges, gender, Contract, PaymentMethod), alongside the target column **Churn**. No complex indexing or schema relationship models are implemented.

9. Machine Learning Pipeline

The machine learning stack relies entirely on classical supervised classification algorithms from Scikit-learn:

- **Logistic Regression:** Achieved the highest F1-Score of 0.6272 (Accuracy: 81.69%) and was saved as the primary model.
- **Gradient Boosting Classifier:** Achieved an F1-Score of 0.5861 (Accuracy: 80.55%).
- **Random Forest Classifier:** Achieved an F1-Score of 0.5500 (Accuracy: 79.56%).

Important Note: No Large Language Models (LLMs), RAG systems, BERT, transformers, vector databases, embedding generators, deep neural networks (PyTorch/TensorFlow), or AI agents are used in this project.

10. Deployment Configuration

The application is prepared for deployment on **Render** (or equivalent WSGI PaaS):

- **Web Server:** Gunicorn serves the Flask WSGI app via web: gunicorn app:app configured in the Procfile.
- **Runtime:** Set to Python version python-3.12.6 in runtime.txt.
- **Build Process:** Auto-installs dependencies declared in requirements.txt.
- **Environment Variables:** No environment variables or configuration files are implemented in this codebase.

11. Key Technical Learnings

- **ETL Pipeline Isolation:** Separated data preprocessing and schema definitions from model training, keeping code clean.
- **Stateless API Serving:** Real-time inference relies on frozen checkpoints (model, features, encoders) loaded into memory, rendering the API stateless.
- **Path Portability:** Using dynamic pathing based on script location guarantees portability across OS and container setups.
- **Robust API Validation:** Implementing error handling (checking client parameters, casting, and catching value exceptions) avoids 500 errors.

12. Challenges Faced & Solutions

- **Dirty Numeric Fields:** The TotalCharges column contained blank spaces that caused type conversion errors. Solved by setting errors='coerce' in Pandas to convert them to NaNs, then imputing them with the median computed from the training data.
- **Train-Serve Skew:** Categorical mapping indexes could shift if encoders were fit on the fly. Solved by saving the Scikit-learn LabelEncoder objects from the preprocessing script to encoders.pkl and loading them directly inside the API.

13. Resume Support Materials

ATS Project Description: Developed a Customer Churn Prediction System that cleans raw subscriber data, stores records in SQLite, evaluates three Scikit-learn classifiers, and serves predictions in real-time. The best performing Logistic Regression model is exposed via a stateless Flask REST API configured with Gunicorn, inputs validation, and relative pathing, establishing a highly portable, production-ready microservice.

Core Resume Points:

- Developed an end-to-end churn prediction pipeline, training and comparing Logistic Regression, Random Forest, and Gradient Boosting classifiers in Scikit-learn, achieving an F1-score of 0.6272 and an accuracy of 81.69% on validation sets.
- Designed and deployed a stateless Flask REST API with Gunicorn for concurrency, implementing inputs validation schemas that catches missing fields and returns detailed client-side HTTP 400 errors.
- Resolved train-serve skew by serializing preprocessing LabelEncoder objects and feature column ordering templates to pickle files, ensuring consistent real-time encoding during API predictions.

Key Skills: Python, Pandas, NumPy, Scikit-learn, SQLite, Flask, Joblib, Gunicorn, REST API development, Data Imputation, Feature Serialization.

14. Comprehensive Interview Preparation

Q: Why did you choose the F1-Score as your champion selection metric?

A: The dataset is imbalanced. Accuracy can look high but miss all minority-class churn cases. F1-Score is the harmonic mean of precision and recall, balancing our false alerts vs. missed cases.

Why they ask: Shows understanding of model evaluation trade-offs in real-world imbalanced datasets.

Q: What happens to your API if a user sends a category value that wasn't in the training set?

A: The API catches it by checking the classes in the loaded LabelEncoder. It returns a 400 Bad Request, detailing the invalid category value and listing valid categories.

Why they ask: Demonstrates defensive programming and error boundary checks in REST API engineering.

Q: Is SQLite appropriate for concurrent prediction requests in production?

A: SQLite is only used during training. The Flask API runs completely statelessly in-memory and does not query SQLite at all during inference, eliminating database lock bottlenecks.

Why they ask: Checks system design understanding and isolates database requirements from API runtime latency.

15. Candidate Elevator Pitches

30-Second Elevator Pitch:

'I built a Customer Churn Prediction System that helps subscription-based businesses identify customers at risk of canceling their services. The project consists of a Python preprocessing pipeline, a training engine evaluating three Scikit-learn classifiers, and a REST API served via Flask and Gunicorn that provides real-time predictions and probabilities. The system is completely portable and stateless, resolving all paths relative to the script directory.'

3-Minute Pitch:

'I developed a Customer Churn Prediction System featuring a REST API, aimed at solving retention problems in subscription businesses. The project is split into three main components: a preprocessing pipeline, a training and evaluation script, and a web API.

In the Preprocessing stage, I focus on data quality. I load raw Telco Churn CSV data, drop customerID, and convert TotalCharges to numeric, imputing NaNs with the median. I then map the target variable to 1 and 0, fit Scikit-learn LabelEncoders to 15 categorical features, and serialize these encoders using joblib to reuse them during API inference. The clean data is written to SQLite.

In the Model Training stage, I pull the cleaned dataset from SQLite, split the features, and perform an 80/20 train/test split. I train Logistic Regression, Random Forest, and Gradient Boosting. I compare accuracy, precision, recall, and F1-score. Since churn datasets are imbalanced, F1-score is my selection metric. Logistic Regression came out on top with an F1-score of 0.6272 and an accuracy of 81.69%. The script automatically saves this model and the feature column order.

In the Serving stage, I built a REST API using Flask. When the server starts up, it loads the model, feature sequence, and pre-fitted encoders. The predict endpoint accepts a POST request, validates the payload, transforms categorical inputs using the loaded encoders, re-aligns them, and returns predictions and probabilities. For production deployment, I installed Gunicorn as the WSGI server, created a Procfile and runtime.txt, and configured dynamic relative paths. This ensures the entire system is portable and ready to deploy on Render.'